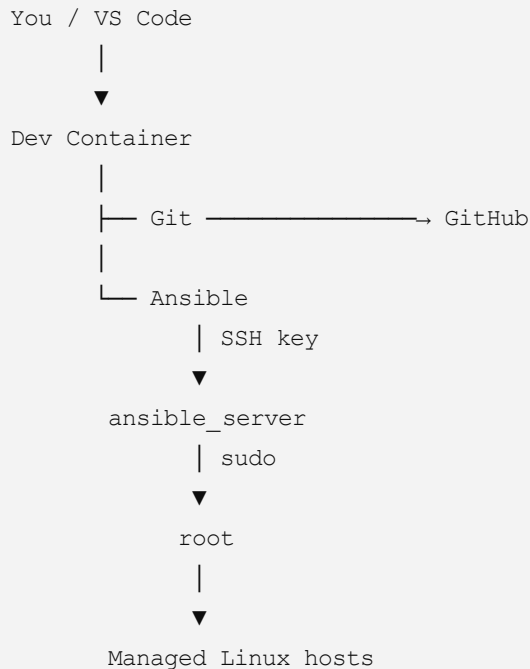


Absolutely. Today covered **much more than “cleaning Git.”** You worked through how an Ansible control environment, SSH authentication, privilege escalation, inventory organization, Git hygiene, and Git history all fit together.

The big picture is:



## 1. We standardized the Ansible automation account

Originally, different machines were being accessed with different users and passwords in inventory. We moved toward one dedicated automation identity:

```
ansible_server
```

on:

```
rhel-dev01
rhel-dev02
servera
serverb
```

The important concept is **separation of identities**.

Your human account is `systemadmin`. The automation account is `ansible_server`. Ansible connects as `ansible_server`, then elevates privileges when necessary.

We verified SSH manually with commands such as:

```
Bash
```

```
ssh -i ~/.ssh/id_ed25519 ansible_server@10.10.10.12
ssh -i ~/.ssh/id_ed25519 ansible_server@10.10.10.4
ssh -i ~/.ssh/id_ed25519 ansible_server@10.10.10.8
```

The mental model is:

```
SSH authentication:
"Can ansible_server log into this machine?"

Privilege escalation:
"Once logged in, can ansible_server become root?"
```

Those are **two completely separate authentication problems**.

---

## 2. We figured out which SSH keys belonged to whom

This was one of the more confusing parts.

We used:

Bash

```
ssh-keygen -lf ~/.ssh/id_ed25519.pub
```

and:

Bash

```
ssh-keygen -lf ~/.ssh/authorized_keys
```

to compare key **fingerprints**.

A fingerprint is basically a short identifier for a key. Instead of visually comparing huge public-key strings, you compare:

```
SHA256:xxxxxxxxxxxxxxxxxxxx
```

We discovered that the key installed in `authorized_keys` matched the Ansible/dev-container key where expected.

We also verified SSH permissions:

Bash

```
stat -c '%U:%G %a %n' \
/root/.ssh \
```

```
/root/.ssh/ansible_lab \  
/root/.ssh/ansible_lab.pub
```

which gave:

```
/root/.ssh          700  
ansible_lab        600  
ansible_lab.pub     644
```

Those permissions make sense:

```
.ssh directory    700 → only owner can access  
private key       600 → only owner can read/write  
public key        644 → others may read it
```

The **private key is the secret**. The `.pub` file isn't.

---

### 3. We diagnosed an SSH-agent problem

At one point:

```
Bash  
ssh-add -l
```

failed for `ansible_server`.

We discovered:

```
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
```

but `/run/user/1000` belonged to `systemadmin`.

That taught an important lesson:

An SSH agent belongs to a user/session. Merely having the `SSH_AUTH_SOCK` environment variable doesn't grant another user access to that agent.

Later this became useful because your **systemadmin SSH agent is forwarded into the dev container**, which is how we ultimately authenticated to GitHub without copying your personal private key into the container.

---

## 4. We configured passwordless privilege escalation

SSH worked, but Ansible failed when `become` was enabled.

That was because:

|                     |   |
|---------------------|---|
| SSH authentication  | ✓ |
| sudo authentication | ✗ |

`ansible_server` could log in, but `sudo` wanted a password.

We checked:

Bash

```
sudo -l
```

and:

Bash

```
sudo whoami
```

Initially:

```
ansible_server
|
| sudo + PASSWORD
▼
root
```

We created a dedicated sudoers drop-in:

```
/etc/sudoers.d/ansible_server
```

containing:

```
ansible_server ALL=(ALL) NOPASSWD: ALL
```

The important architecture decision was **not modifying the entire wheel group**.

Changing wheel would affect every administrator in that group. A dedicated drop-in applies the privilege specifically to the automation account.

We validated sudo configuration with:

Bash

```
visudo -c
```

and learned that sudoers files should have:

```
root:root  
0440
```

We corrected that with:

Bash

```
chmod 0440 /etc/sudoers.d/ansible_server
```

This is why your final automation path can be noninteractive:

```
Ansible  
  |  
  | SSH key  
  ▼  
ansible_server  
  |  
  | NOPASSWD sudo  
  ▼  
root
```

---

## 5. We automated the automation account configuration

Instead of manually configuring every host, you used:

```
playbooks/assign_ansible_server_user.yml
```

to create/configure the account and deploy its sudoers configuration.

You practiced check mode:

Bash

```
ansible-playbook -C playbooks/assign_ansible_server_user.yml -b -K
```

`-C` means:

Tell me what would change without actually changing it.

Then you ran the real playbook.

An especially important lesson came from the second execution.

The playbook reported things like:

```
ok
changed
```

That exposed the concept of **idempotence**.

Ideally:

```
first run
changed → changed → changed

second run
ok → ok → ok
```

because the desired state already exists.

---

## 6. We redesigned the inventory

Your old inventory repeated connection information and contained passwords.

You moved toward:

```
INI

[centos]
centos_servera ansible_host=10.10.10.4
centos_serverb ansible_host=10.10.10.8

[rhel]
rhel-dev01 ansible_host=10.10.10.11
rhel-dev02 ansible_host=10.10.10.12

[remote_log_server]
rhel-dev02

[remote_log_clients]
centos_servera
centos_serverb
rhel-dev01

[linux:children]
centos
```

```
rhel

[linux:vars]
ansible_user=ansible_server
...
```

The key concept is **inheritance**.

Instead of writing:

```
servera → ansible_user=ansible_server
serverb → ansible_user=ansible_server
dev01   → ansible_user=ansible_server
dev02   → ansible_user=ansible_server
```

you say:

```
linux
├── centos
│   ├── servera
│   └── serverb
└── rhel
    ├── dev01
    └── dev02

linux:vars
    ansible_user=ansible_server
```

All Linux hosts inherit it.

We verified the hierarchy with:

Bash

```
ansible-inventory --graph
```

That produced the exact group tree Ansible sees.

---

## 7. We validated `ansible.cfg`

We used:

Bash

```
ansible-config dump --only-changed
```

This is much better than merely running:

Bash

```
cat ansible.cfg
```

because `cat` tells you what **you wrote**.

`ansible-config dump` tells you what **Ansible actually loaded**.

We verified things such as:

```
DEFAULT_BECOME = True
DEFAULT_BECOME_ASK_PASS = False
DEFAULT_BECOME_USER = root
DEFAULT_HOST_LIST = inventory
HOST_KEY_CHECKING = False
```

That explains why this eventually worked:

Bash

```
ansible linux -m ping
```

without:

Bash

```
-b
-K
```

because the configuration already told Ansible:

```
become = true
become_ask_pass = false
```

Then all four systems returned:

```
pong
```

That was the final end-to-end validation.

---

## 8. We cleaned the repository structure

Your repository originally mixed scripts, configuration payloads, notes, logs and playbooks.

We reorganized toward:

```
rhel-engineer-home-lab/
|
├─ .devcontainer/
|
├─ docs/
|   └─ playbook_notes.txt
|
├─ files/
|   ├── ansible_server
|   ├── remote_logger.conf
|   └─ remote_logs.conf
|
├─ playbooks/
|   ├── assign_ansible_server_user.yml
|   ├── host_log_inspector.yaml
|   └─ remote_log_server_config.yml
|
├─ scripts/
|   └─ day1_breakfix.sh
|
├─ .gitignore
├─ ansible.cfg
├─ ansible-navigator.yml
└─ inventory
```

The logic is:

```
playbooks/ → automation instructions
files/      → files Ansible deploys
scripts/    → utilities you execute
docs/       → human documentation
```

That's much easier for another administrator to understand.

---

## 9. We fixed portable Ansible file paths

You temporarily changed playbook `src:` paths to absolute paths.

Something like:

```
/workspaces/rhel-engineer-home-lab/files/remote_logger.conf
```

works, but ties the playbook to that exact machine/container.

We changed the approach so the playbook uses repository-relative paths.

The principle is:

```
Absolute filesystem path
    ↓
works on MY controller

Relative/project path
    ↓
works from the cloned project
```

That's important for Git-based automation.

---

## 10. We created `.gitignore`

You built:

gitignore

```
# Runtime logs
*.log
*.retry

# SSH/private credentials
*.pem
*.key
id_rsa
id_ed25519

# Ansible secrets
vault_password*
*.vault-pass

# Editor/temp files
*~
*.swp
```

We tested the rule with:

Bash

```
git check-ignore -v ansible-navigator.log
```

Git responded with the exact rule responsible:

```
.gitignore:2:*.log ansible-navigator.log
```

That taught an important distinction:

`.gitignore` prevents untracked files from being added. It does **not** automatically remove files Git already tracks.

---

## 11. You learned `git rm --cached`

`ansible-navigator.log` was already tracked.

You looked at the Git documentation and found:

Bash

```
git rm --cached ansible-navigator.log
```

The important distinction:

```
git rm file
  ↓
remove from Git
AND
remove from disk

git rm --cached file
  ↓
remove from Git
BUT
keep local file
```

That was a good documentation-driven task because you found `--cached` yourself.

---

## 12. We learned how Git sees moves

Initially Git showed:

```
deleted: day1_breakfix.sh
untracked: scripts/

deleted: playbook_notes.txt
untracked: docs/
```

After staging everything with:

Bash

```
git add -A
```

Git recognized:

```
playbook_notes.txt => docs/playbook_notes.txt  
day1_breakfix.sh => scripts/day1_breakfix.sh
```

Git doesn't fundamentally store a special "move file" operation. It examines the content and determines that a deletion plus addition are similar enough to represent as a rename.

---

## 13. We reviewed changes before committing

We used:

Bash

```
git status
```

then:

Bash

```
git diff --cached --stat
```

and:

Bash

```
git diff --cached
```

Think of these as:

```
git status  
  ↓  
WHAT changed?  
  
git diff --cached --stat  
  ↓  
HOW MUCH changed?  
  
git diff --cached  
  ↓  
EXACTLY WHAT am I about to commit?
```

That's a strong pre-commit workflow.

## 14. We discovered a serious Git security problem

The current inventory no longer contained passwords—but this:

Bash

```
git diff --cached
```

showed deleted lines containing old `ansible_become_pass` values.

That meant:

|              |                            |
|--------------|----------------------------|
| Current file | → clean                    |
| Git history  | → still contains passwords |

This is critical:

Deleting a secret from a Git file does not delete the secret from old commits.

Git remembers previous versions.

---

## 15. We installed and used `git-filter-repo`

First we checked:

Bash

```
git filter-repo --version
```

Initially it wasn't installed.

After installation we got:

```
473e7df3dd02
```

Then we created replacement rules outside the repository:

```
/tmp/replacements.txt
```

Conceptually:

```
literal:<old-secret>==>[REDACTED]
```

We protected it:

Bash

```
chmod 600 /tmp/replacements.txt
```

Before rewriting anything, we also made a Git bundle backup:

Bash

```
git bundle create /tmp/rhel-engineer-home-lab-before-cleanup.bundle --all
```

and verified it:

Bash

```
git bundle verify /tmp/rhel-engineer-home-lab-before-cleanup.bundle
```

Then came the major operation:

Bash

```
git filter-repo --replace-text /tmp/replacements.txt --force
```

That walked through **all eight commits** and rewrote historical file contents.

Before:

```
commit A → password  
commit B → password  
commit C → password
```

After:

```
new commit A → [REDACTED]  
new commit B → [REDACTED]  
new commit C → [REDACTED]
```

---

## 16. Why all the Git commit hashes changed

Before the rewrite you had hashes such as:

```
0f72115  
814cfea
```

740~052

Afterward:

```
bc6f102
dc3472e
2a5c302
```

That's expected.

A Git commit ID is derived from the commit's contents and metadata, including its parent relationship.

Change historical content and you effectively create a new chain:

OLD

```
A — B — C — D
                ↑
              main
```

REWRITE

```
A' — B' — C' — D'
                ↑
              main
```

`A'` through `D'` represent sanitized versions of the history.

---

## 17. We verified the secrets were actually gone

We searched every reachable commit:

Bash

```
git grep 'OLD_PASSWORD' $(git rev-list --all)
```

You ran this for the actual old values and received:

```
no output
```

That's exactly what we wanted.

Meanwhile searching the historical variable showed:

```
ansible_become_pass=[REDACTED]
```

So we proved:

|                                     |   |
|-------------------------------------|---|
| Historical configuration preserved  | ✓ |
| Actual historical passwords removed | ✓ |

---

## 18. Why `origin` disappeared

After `git-filter-repo`, this happened:

```
NOTICE: Removing 'origin' remote
```

That's deliberate safety behavior.

A history-rewriting tool doesn't want you accidentally doing:

```
rewrite history
↓
automatically push
↓
destroy remote history
```

So we deliberately restored it afterward:

Bash

```
git remote add origin git@github.com:jeffthompsonjr1/rhel-engineer-home-lab.git
```

and verified:

Bash

```
git remote -v
```

---

## 19. Then GitHub SSH failed

When we tried:

Bash

```
git push --force origin main
```

GitHub responded:

```
Permission denied (publickey)
```

We diagnosed it with:

Bash

```
ssh -vT git@github.com
```

The key line was:

```
Offering public key:
systemadmin@rhel-dev01.lab.example.com
```

This proved the dev container was receiving the `systemadmin` SSH key through your SSH agent.

GitHub simply didn't recognize that public key yet.

Once you registered the public key, we tested:

Bash

```
ssh -T git@github.com
```

and got:

```
Hi jeffthompsonjr1!
You've successfully authenticated...
```

That gave us another useful architecture:

```
rhel-dev01
systemadmin private key
    |
    | SSH agent
    ▼
Dev container
    |
    | key used without copying private key
    ▼
GitHub
```

That's preferable to stuffing your personal GitHub private key into a disposable container.

## 20. Finally, we force-pushed the sanitized history

We ran:

Bash

```
git push --force origin main
```

GitHub reported:

```
+ 7d8c053...bc6f102 main -> main (forced update)
```

A normal push says:

Add my new commits after what's already there.

A force push says:

Make the remote branch point to **my version of history**, even though its history differs from the current remote.

Normally force pushing deserves caution.

Here it was intentional because:

```
GitHub
OLD history containing passwords
      ↓
    REPLACE
      ↓
Local
SANITIZED history
```

---

## The commands worth remembering

You don't need to memorize every command from tonight. I'd concentrate on these:

Bash

```
# Understand Ansible configuration
ansible-config dump --only-changed

# Understand inventory structure
ansible-inventory --graph
```

```
# Test fleet connectivity
ansible linux -m ping

# Validate sudo configuration
visudo -c
sudo -l

# Inspect SSH keys/fingerprints
ssh-keygen -lf <public-key>

# Troubleshoot SSH
ssh -v user@host

# Test GitHub SSH
ssh -T git@github.com

# Repository state
git status

# See tracked files
git ls-files

# Stop tracking without deleting locally
git rm --cached <file>

# Determine why something is ignored
git check-ignore -v <file>

# Stage additions/modifications/deletions
git add -A

# Review staged changes
git diff --cached
git diff --cached --stat

# Search history
git log -S'<text>' --all --oneline

# List all reachable commits
git rev-list --all

# Search all reachable historical versions
git grep '<text>' $(git rev-list --all)

# Rewrite sensitive history
git filter-repo --replace-text <replacement-file> --force

# Inspect remotes
git remote -v
```

**The five concepts I most want you to retain**

If you remember nothing else from tonight, remember these:

### 1. SSH authentication and sudo/become are separate layers.

```
SSH → Can I log in?  
sudo → Can I become root?
```

### 2. Ansible inventory inheritance prevents repetition.

```
linux:vars  
  ↓  
all children of linux inherit the setting
```

### 3. `.gitignore` doesn't remove something already tracked.

For that you learned:

Bash

```
git rm --cached
```

### 4. Deleting a password from the current file does not remove it from Git history.

That's why we needed:

Bash

```
git filter-repo
```

### 5. Validate the effective state, not merely the configuration file.

Instead of assuming:

```
"I wrote the config correctly."
```

you used:

Bash

```
ansible-config dump --only-changed  
ansible-inventory --graph  
ansible linux -m ping  
visudo -c  
ssh -T git@github.com  
git grep ... $(git rev-list --all)
```

Each command answered:

## Is the system actually behaving the way I intended?

That's probably the most important sysadmin habit from the entire session.

For your mastery tracker, I would count tonight as substantial exposure to **Git repository administration, SSH identity management, Ansible inventory/configuration, and privilege escalation**, but keep SSH/become around **50% mastery** until you can reproduce this workflow using documentation with little or no assistance.